LAB2_315551006_劉天恩

1. Introduction (20%)

Electroencephalography (EEG) records electrical activity along the scalp and provides high temporal resolution important for Brain-Computer Interface (BCI) applications. However, decoding neural signals remains difficult due to poor signal to noise ratios (SNR), subject specific variability, data hard to get which makes limited training sample sizes, and expensive.

EEGNet (Lawhern et al., 2018) is a compact Convolutional Neural Network specifically tailored for EEG based BCIs. It jointly optimizes frequency feature extraction, spatial mapping, and category prediction within a unified framework by combining:

- **Temporal Convolutions:** 2D filters spanning time samples per channel to capture frequency dynamics. It ignores where on the scalp the signal is coming from and focuses entirely on when and how fast the signal is changing. The shape of this filter is rectangular 1 pixel high and many pixels wide so that it stretches horizontally to catch the waves regardless of the spatial information.
- **Depthwise Convolutions:** Spatial filters acting independently per temporal feature map, learning optimal channel weights across EEG electrodes without mixing cross-band projections. Spatial filtering identifies which channels(parts of the brain) contribute most effectively to feature extraction. Think about the shape opposite of the temporal filter, where the high has many pixels and wide only 1 pixel such that these filters see the relation between channels. This layer **kept from being fully connected**, which significantly reduces the number of parameters and prevents overfitting
- **Separable Convolutions:** Merges the extracted temporal and spatial information using a separable depthwise convolution and a pointwise filter pointwise (1x1) convolution and reducing parameter counts. This step summarizes all previous feature maps to create a unified representation, which is then fed into a final classification stage to output the predicted class/es.
- With roughly 2,000 to 3,000 trainable parameters, EEGNet minimizes overfitting on limited trial sets while matching or exceeding the performance of significantly larger deep architectures.

2. Experiment Setups and Results (35%)

- **Highest Testing Accuracy:**

```
EEGNet_ReLU_Drop0.5.pt: 100%|          | 700/700 [01:36<00:00,  7.23it/s
--> [EEGNet_ReLU_Drop0.5] Best Peak Test Acc: 88.33% (at Epoch 594/700)
```

- **Experiments Conducted:**

Experiment	Model	F1-D-F2	Kernel	Act	Dropout	Best Epoch	Max Train Acc (%)	Peak Test Acc (%)
EEGNet_ReLU_Drop0.5	EEGNet	16-2-32	51	relu	0.5	594/700	95.6	**88.33**
Custom_EEGNet_Drop0.50_0.50	EEGNet	16-2-32	51	leaky_relu (0.05)	(0.30, 0.50)	421/501	94.9	**87.87**
Custom_EEGNet_Drop0.30_0.50	EEGNet	8-2-16	64	leaky_relu (0.05)	(0.30, 0.50)	248/298	93.0	**87.31**
Custom_EEGNet_Drop0.25_0.50	EEGNet	8-2-16	64	leaky_relu (0.05)	(0.25, 0.50)	228/278	93.0	**87.04**
EEGNet_LeakyReLU_0.05	EEGNet	16-2-32	51	leaky_relu (0.05)	0.25	345/545	99.8	**86.48**
EEGNet_ReLU	EEGNet	16-2-32	51	relu	0.25	286/486	99.1	**86.39**
Custom_EEGNet_Drop0.25_0.50 (Standard)	EEGNet	16-2-32	51	leaky_relu (0.05)	(0.25, 0.50)	371/451	98.3	**86.20**
EEGNet_ELU_a0.5	EEGNet	16-2-32	51	elu (0.5)	0.25	233/433	99.3	**85.83**
EEGNet_ReLU_Drop0.1	EEGNet	16-2-32	51	relu	0.1	537/700	99.8	**85.09**
DeepConvNet_LeakyReLU_0.10	DeepConvNet	-	(1, 5), (2, 1)	leaky_relu (0.10)	0.5	1256/1676	99.4	**84.54**
Custom_EEGNet_Drop0.25_0.50 (ELU)	EEGNet	16-2-32	51	elu	(0.50, 0.50)	238/318	93.0	**84.72**
EEGNet_ELU	EEGNet	16-2-32	51	elu	0.25	464/664	99.8	**83.98**
DeepConvNet_ReLU_Drop0.25	DeepConvNet	-	(1, 5), (2, 1)	relu	0.25	868/1288	99.5	**83.24**

Experiment	Model	F1-D-F2	Kernel	Act	Dropout	Best Epoch	Max Train Acc (%)	Peak Test Acc (%)
DeepConvNet_ELU_a0.5	DeepConvNet	-	(1, 5), (2, 1)	elu (0.5)	0.5	353/773	99.1	**83.24**
DeepConvNet_ReLU	DeepConvNet	-	(1, 5), (2, 1)	relu	0.5	1069/1569	98.8	**83.24**
DeepConvNet_LeakyReLU	DeepConvNet	-	(1, 5), (2, 1)	leaky_relu (0.05)	0.5	422/922	98.6	**83.15**
DeepConvNet_ReLU_LR0.0005	DeepConvNet	-	(1, 5), (2, 1)	relu	0.5	675/1095	98.6	**83.15**
EEGNet_ELU_a2.0	EEGNet	16-2-32	51	elu (2.0)	0.25	497/697	99.5	**82.13**
DeepConvNet_ReLU_Drop0.50	DeepConvNet	-	(1, 5), (2, 1)	relu	0.5	251/671	97.1	**82.13**
DeepConvNet_ReLU_LR0.001_NoSched	DeepConvNet	-	(1, 5), (2, 1)	relu	0.5	321/741	97.0	**81.57**
DeepConvNet_ReLU_Drop0.75	DeepConvNet	-	(1, 5), (2, 1)	relu	0.75	284/704	84.7	**81.02**
DeepConvNet_ELU	DeepConvNet	-	(1, 5), (2, 1)	elu	0.5	359/859	98.5	**80.74**

Experiment: Multi-Model Ensemble: Test Accuracy: **87.78%** From averaging top 3 result: The ensemble aggregates predictions from the top three performing variants: **EEGNet_ReLU_Drop0.5**, **Custom_EEGNet_Drop0.30_0.50 (Standard)**, and **Custom_EEGNet_Drop0.30_0.50**.

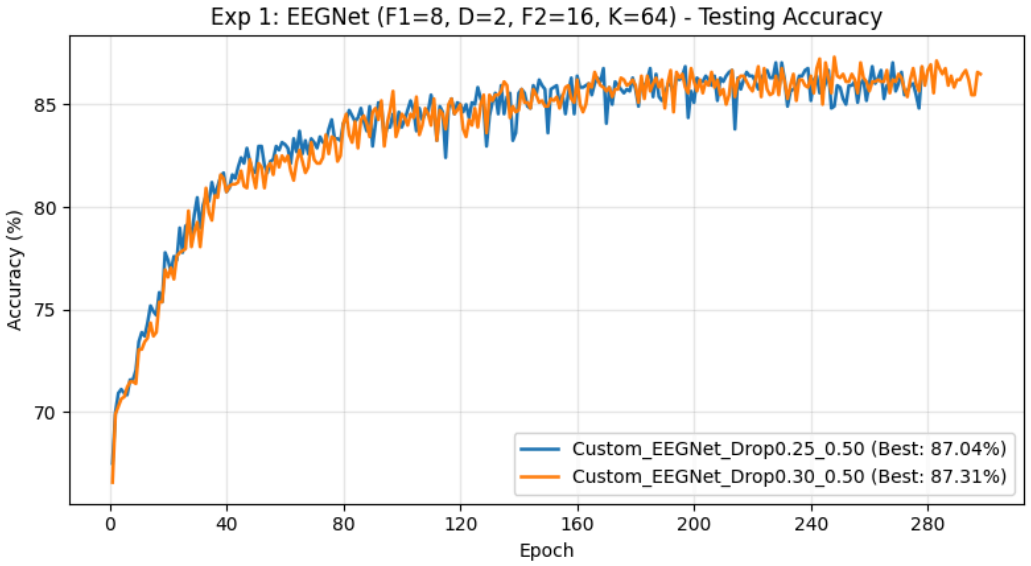
- **Comparison Tables & Plots:**

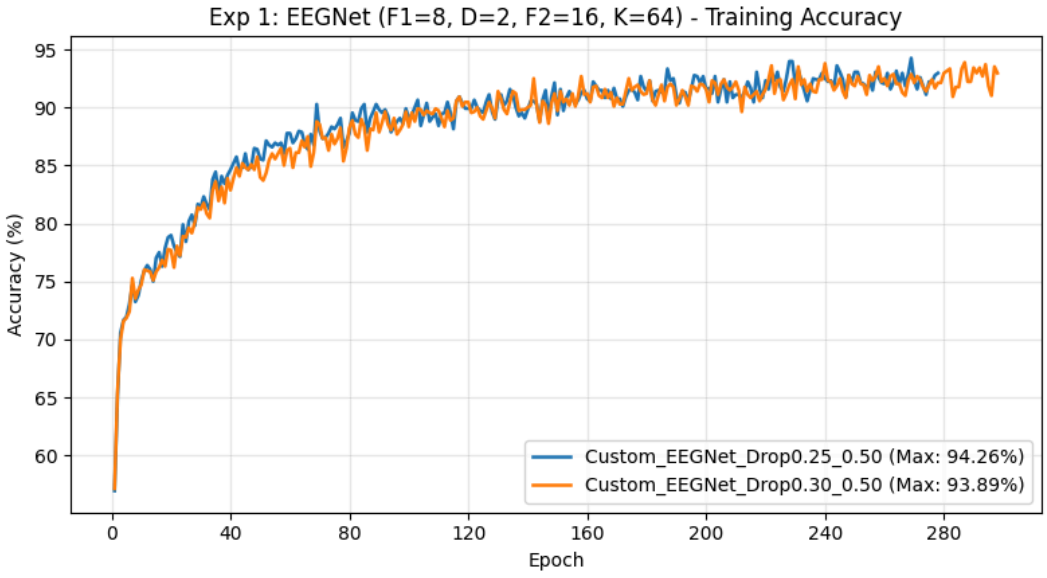
1. EEGNet Stage-Wise Dropout

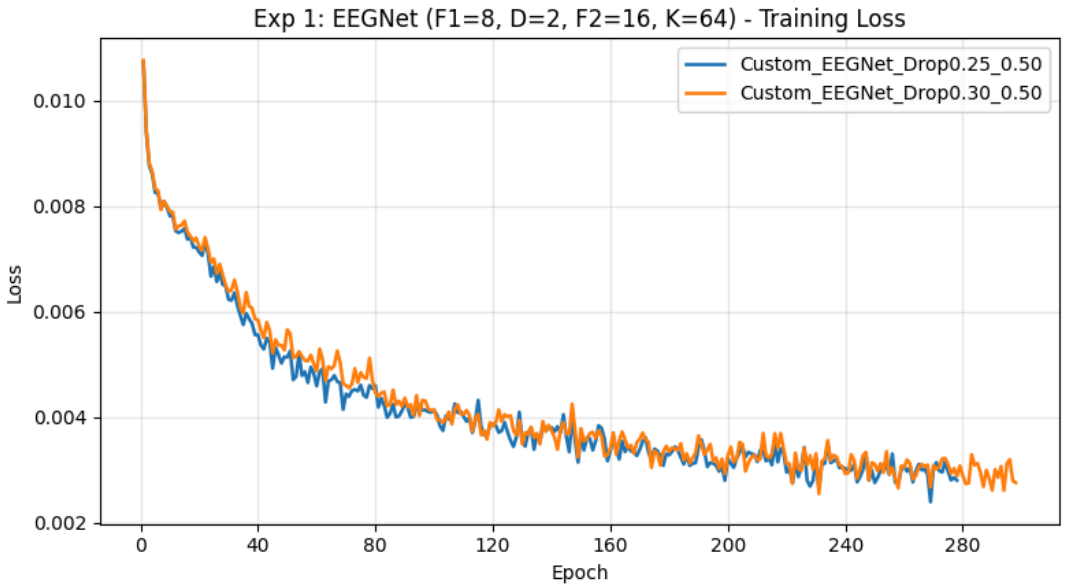
- **Description:** This experiment evaluates a EEGNet architecture by halving the filter bank (F1=8, D=2,F2=16) with kernel (K=64) to prevent feature bottlenecking. It compares two

stage-wise dropout configurations using LeakyReLU to balance low level spatial preservation with high-level regularization.

- **Configurations in Plot:**





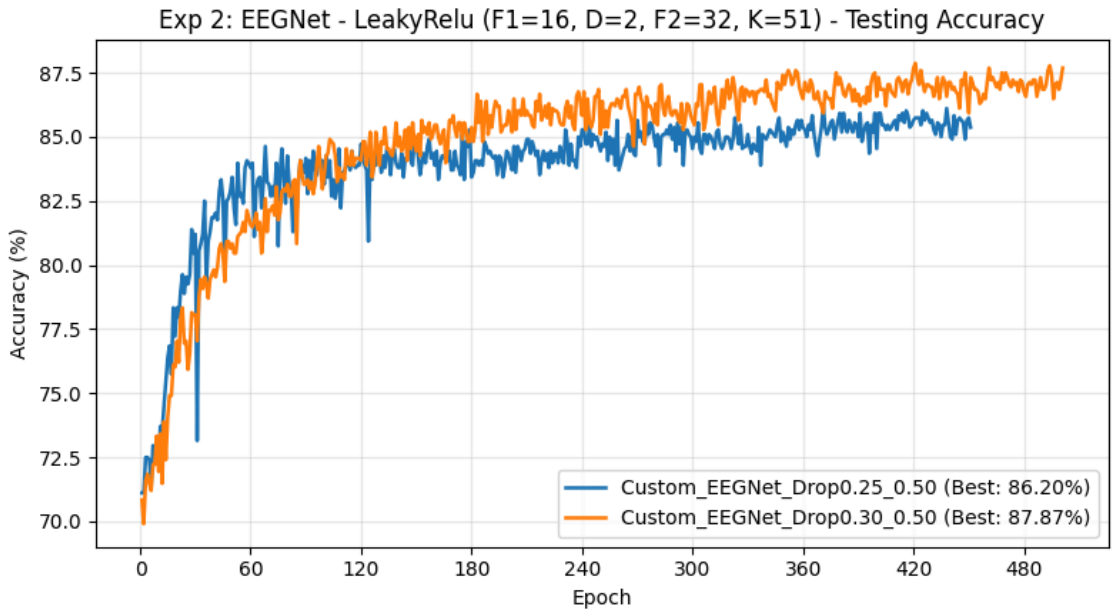


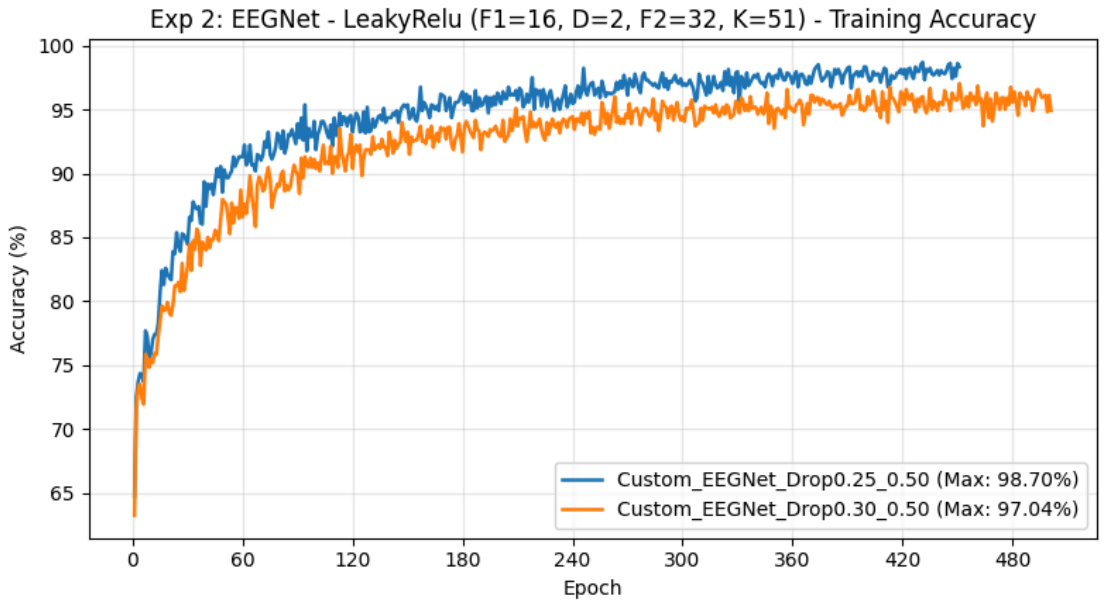
2. Custom Standard EEGNet Stage-Wise Dropout

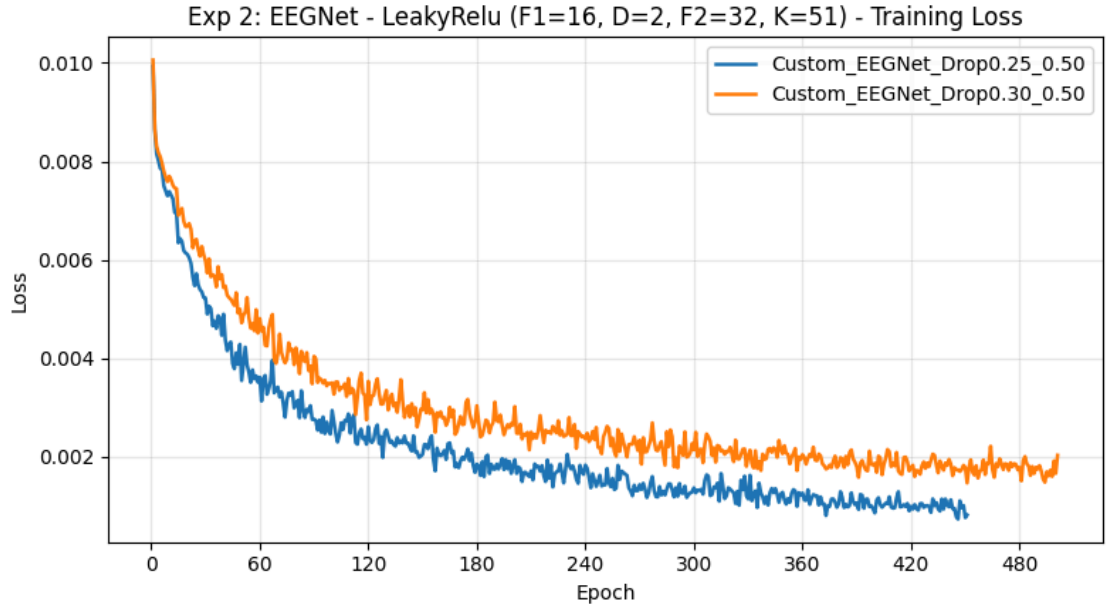
- **Description:** This experiment applies heavier stage-wise dropout to the standard EEGNet filter size (F1=16, D=2,F2=32, K=51) using LeakyReLU. The goal is to determine if heavier early dropout (0.50) performs better than standard light dropout (0.25) before the separable convolution block.

  * Note: The Custom_EEGNet_Drop0.30_0.50 is supposed to be Custom_EEGNet_Drop**0.50_0.50**

- **Configurations in Plot:**

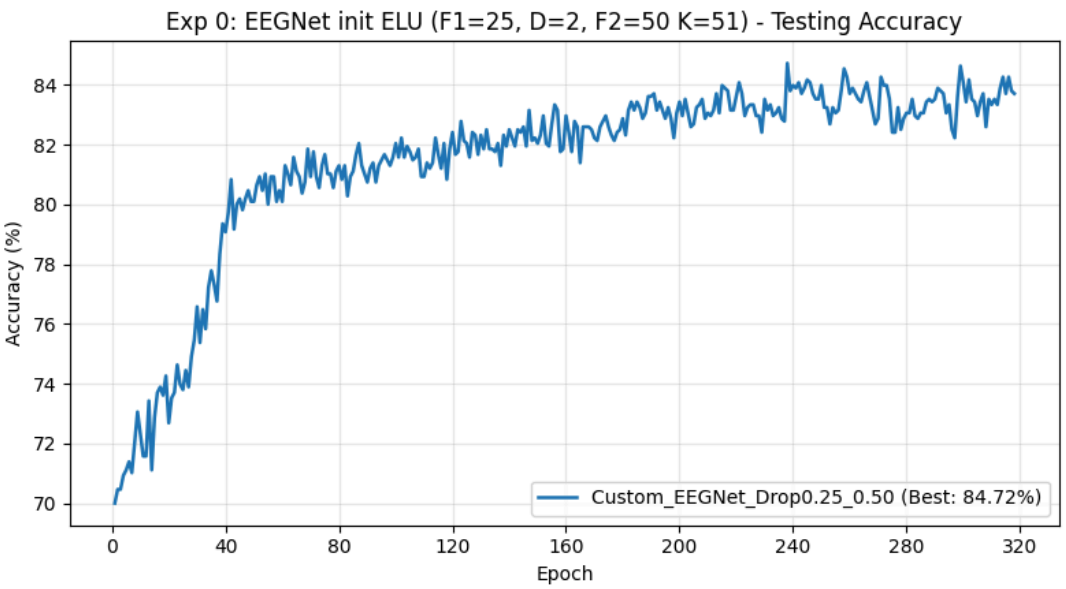


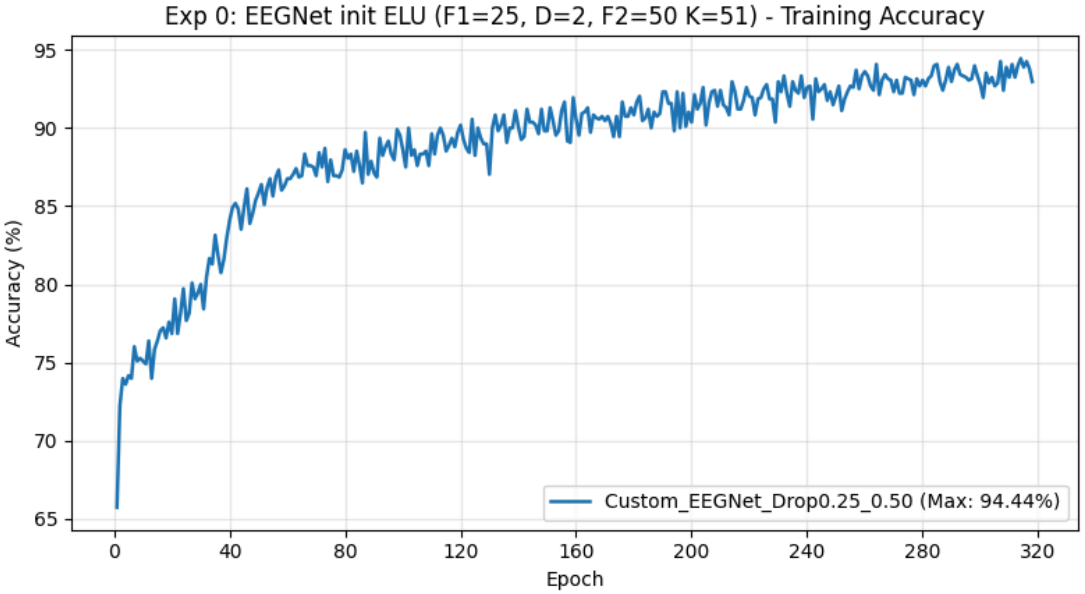


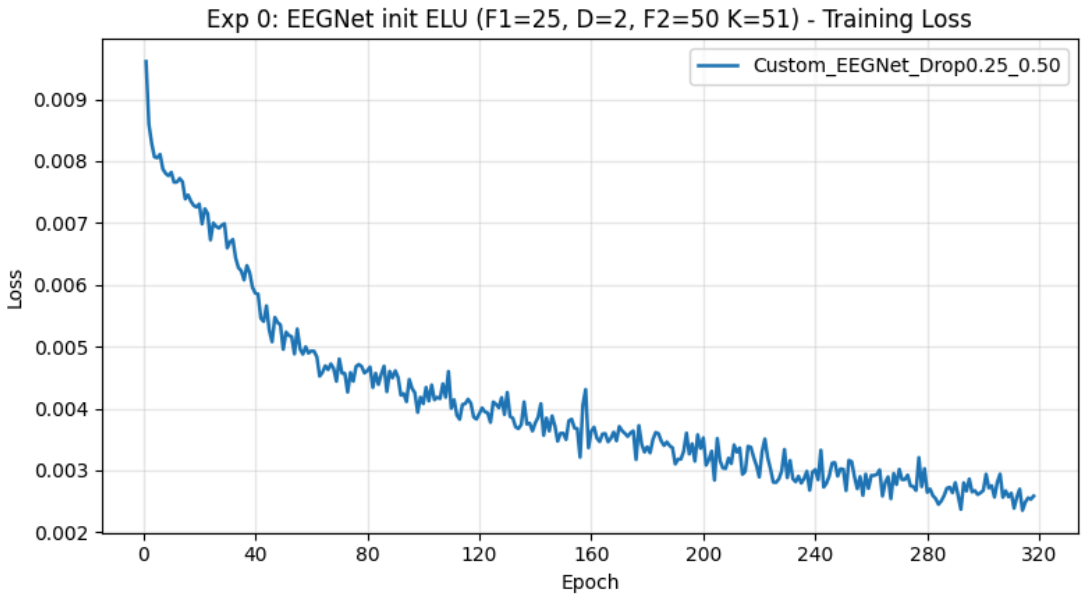


3. EEGNet Lab2 Specs

- **Description:** A standalone test applying aggressive uniform dropout (0.50, 0.50) to the standard EEGNet architecture (F1=16, F2=32) while utilizing the ELU activation function to test noise saturation limits.
- **Configurations in Plot:**

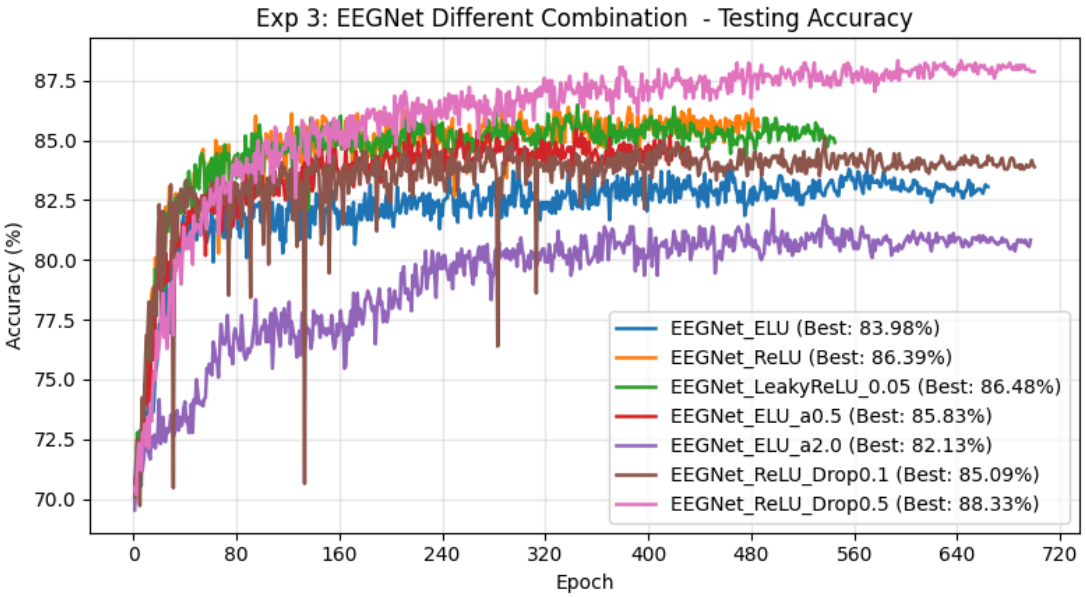

Exp 0: EEGNet init ELU (F1=25, D=2, F2=50 K=51) - Testing Accuracy
Accuracy (%)
Epoch
Custom_EEGNet_Drop0.25_0.50 (Best: 84.72%)

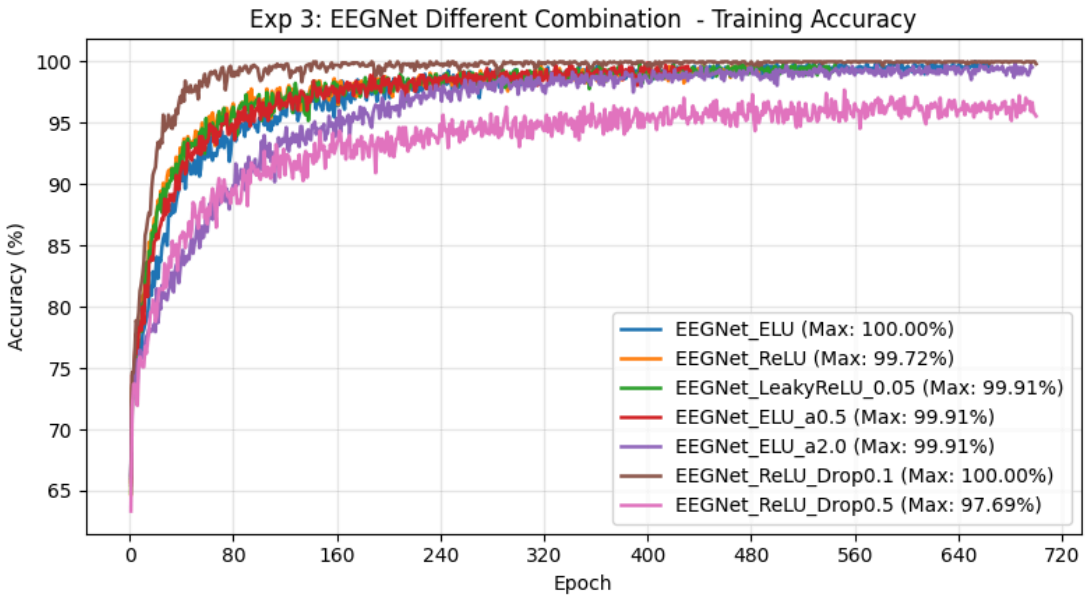

Exp 0: EEGNet init ELU (F1=25, D=2, F2=50 K=51) - Training Accuracy
Accuracy (%)
Epoch
Custom_EEGNet_Drop0.25_0.50 (Max: 94.44%)

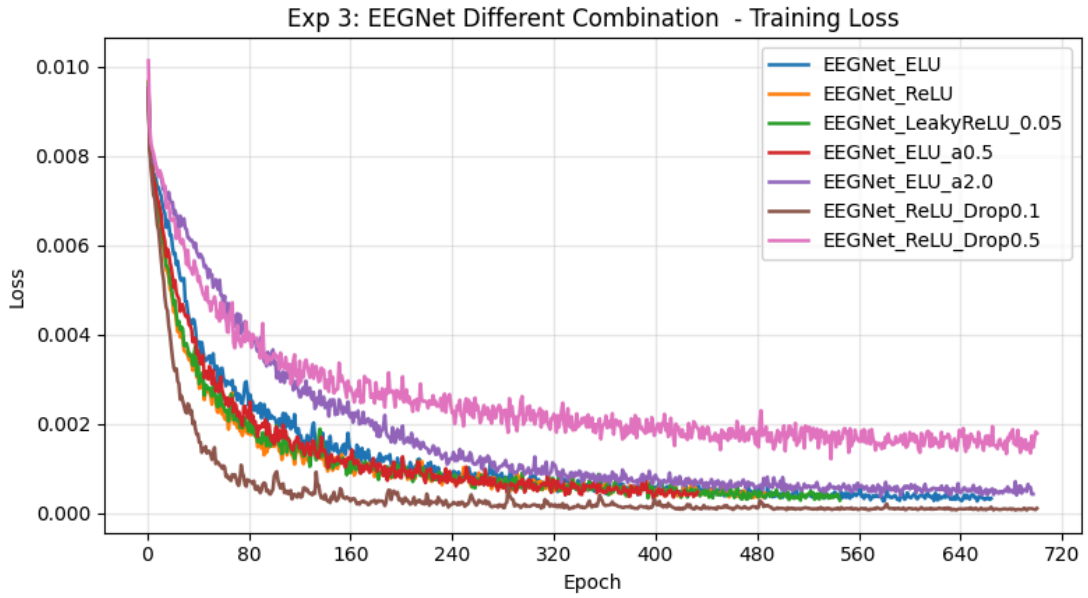


4. Standard EEGNet Variations (Activations, Alpha, Dropout)

- **Description:** Comparing the performance of ELU, ReLU, and LeakyReLU on the baseline EEGNet architecture to determine which activation best suppresses background EEG noise. Evaluating the impact of the ELU negative saturation slope (αα) on classification accuracy. Comparing under-regularized (0.10) and heavily regularized (0.50) uniform dropout rates on the best-performing ReLU architecture.



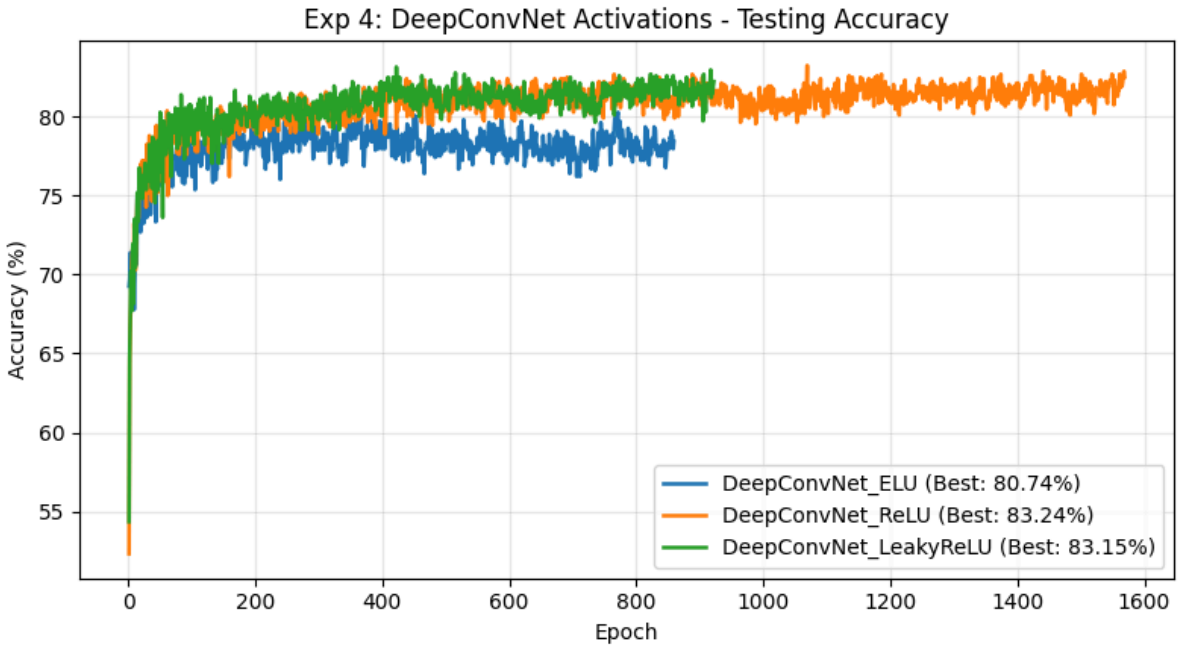


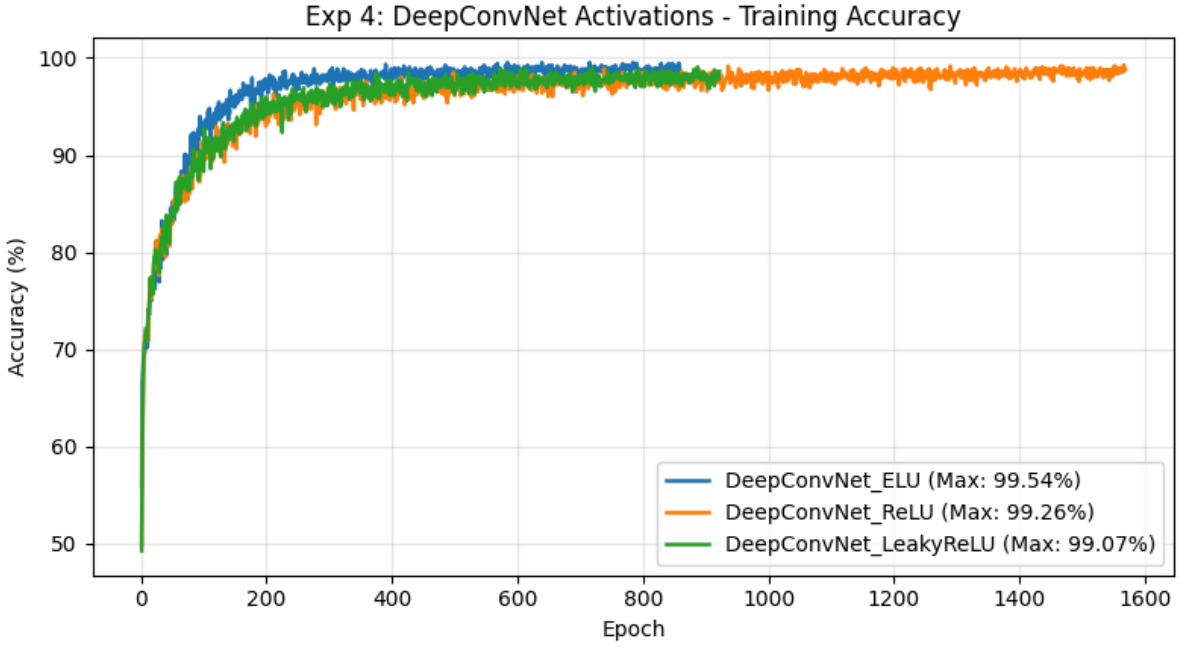


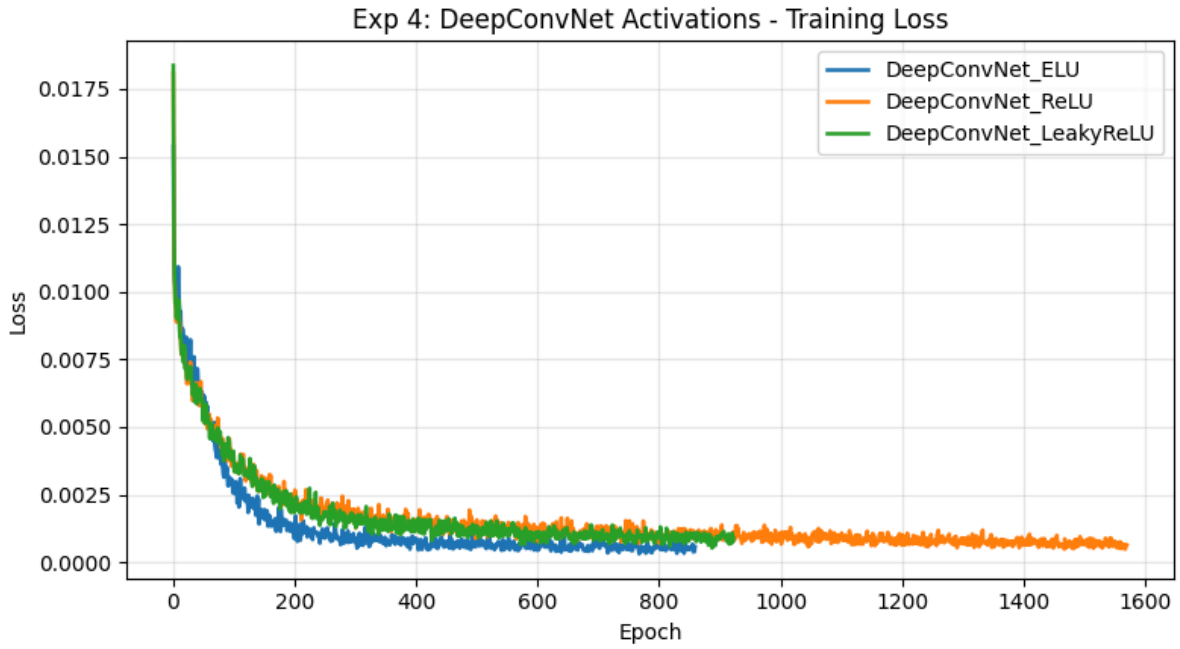
EEGNet_ReLU_Drop0.5 performs the best as the plot shown that it has highest testing while still balancing with the training, compared to other such as EEGNet_Relu_a2.0 that reached training of 99.91% but testing is oscillating which sign of overfitting and it shown in the testing result that is lower (82.13%).

5. DeepConvNet Activation Functions

- **Description:** Comparing ELU, ReLU, and LeakyReLU on the highly parameterized DeepConvNet architecture (~150k parameters) using a standard 0.50 dropout rate to observe how activation sparsity affects a deeper network.


Exp 4: DeepConvNet Activations - Testing Accuracy
Accuracy (%)
Epoch
DeepConvNet_ELU (Best: 80.74%)
DeepConvNet_ReLU (Best: 83.24%)
DeepConvNet_LeakyReLU (Best: 83.15%)

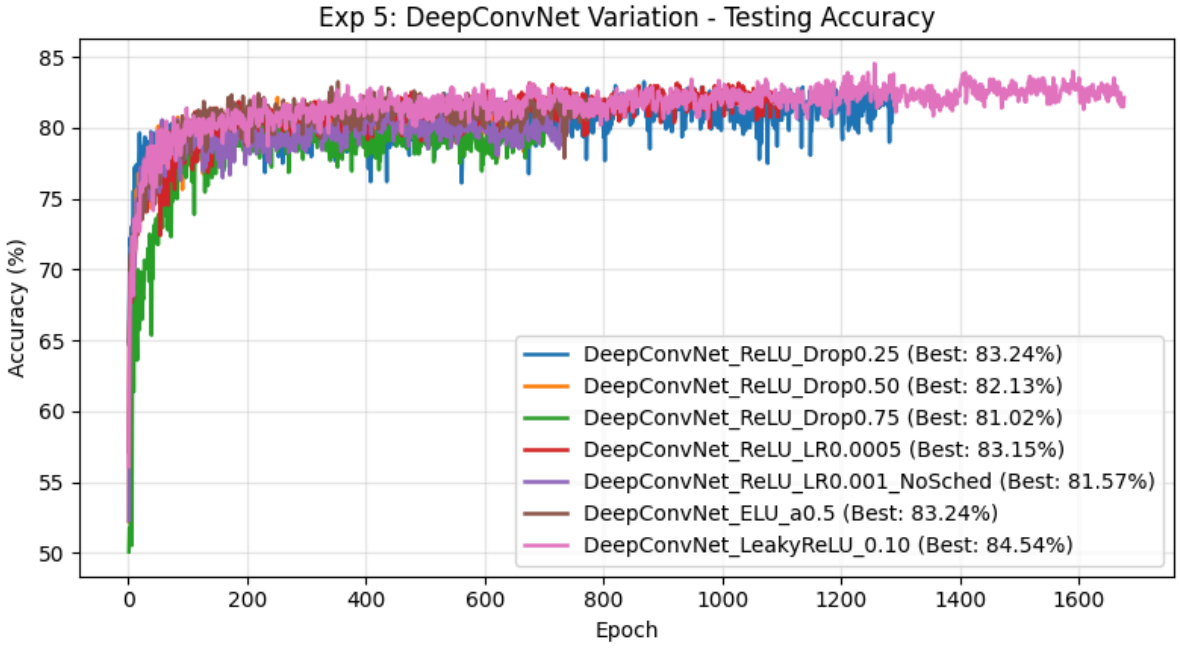

Exp 4: DeepConvNet Activations - Training Accuracy
Accuracy (%)
Epoch
DeepConvNet_ELU (Max: 99.54%)
DeepConvNet_ReLU (Max: 99.26%)
DeepConvNet_LeakyReLU (Max: 99.07%)

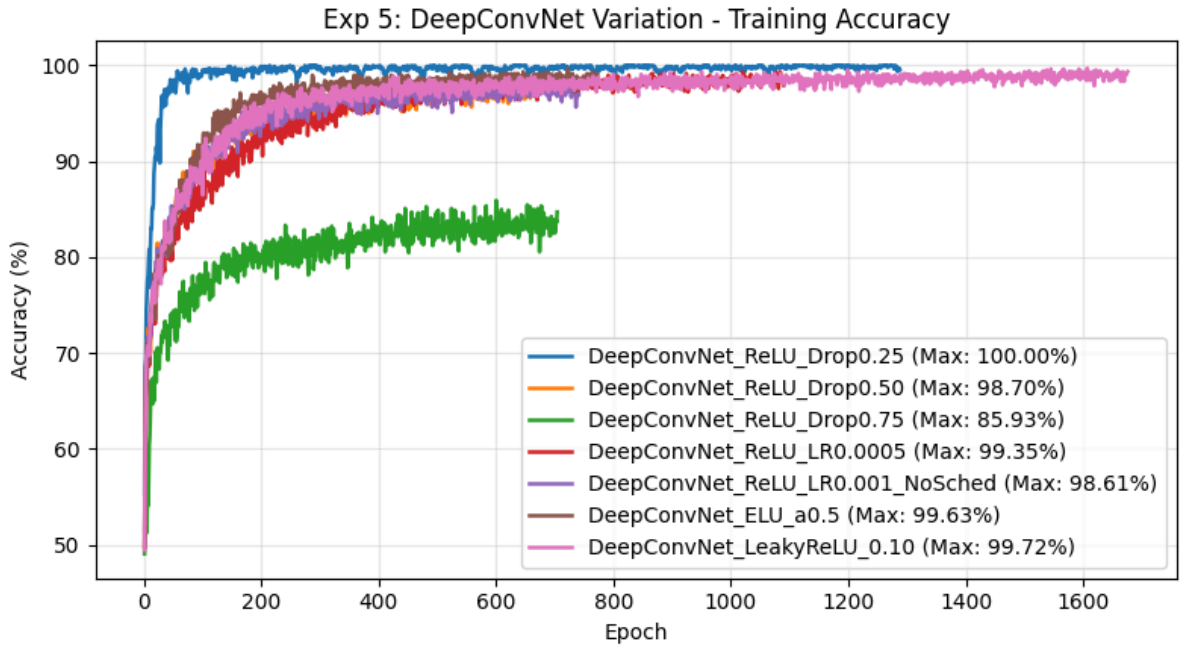


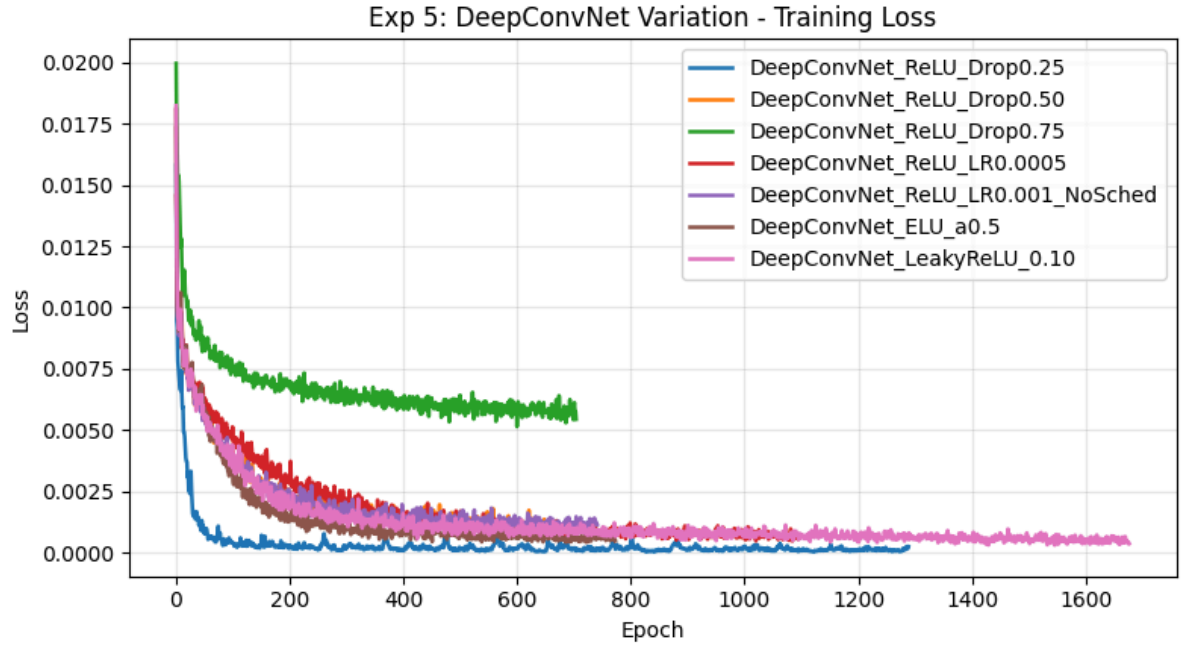
6. DeepConvNet Fine-Tuning (Dropout, LR, Scheduler)

- **Description:** Evaluating the effect of extreme dropout rates (0.25, 0.50, 0.75) to combat the rapid overfitting inherent to DeepConvNet's massive dense classification layer. Investigating training stability by comparing a slower base learning rate (0.0005) and the removal of the Cosine Annealing scheduler. Fine-tuning the negative slope of LeakyReLU (0.10) and the alpha value of ELU (0.5) to optimize noise suppression in the deep convolutional blocks.

  *Note: DeepConvNet_LeakyReLU_0.10, 0.10 is a negative slope.







3. Discussion (40%)

- **Model Implementation:**

1. Training Setup & Global Specifications

These parameters apply to both models during the training phase. This setup follows the Lab2 specs file and follows the format of the data itself including the input and output. Optimizers use ADAM

because it works well for most tasks and can be used as a starting optimizer here and it turned out really well.

Specification	Implementation Details
Input Shape	[Batch Size, 1, 2, 750] (1 Channel, 2 Electrodes, 750 Timepoints)
Output Shape	[Batch Size, 2] (2 Classes: Left vs. Right hand motor imagery)
Loss Function	CrossEntropyLoss ()
Optimizer	Adam (Learning Rate = 1e-3, Weight Decay = 1e-3)
Scheduler	CosineAnnealingLR (T_max = total epochs, eta_min = 1e-5) to smoothly decay the learning rate.
Early Stopping	Checkpoints best_wts via deepcopy at peak test accuracy, stops if no improvement for 50-420 epochs.

2. EEGNet Architecture Breakdown

Total Parameters: ~2,610

Block / Layer	Purpose	PyTorch Implementation & Filter Details
1. FirstConv	Extracts frequency information over time (Temporal Filtering).	Conv2d(in=1, out=16, kernel=(1, 51), padding=(0, 25)) BatchNorm2d(16)
2. DepthwiseConv	Learns spatial channel correlations independently for each temporal filter.	Conv2d(in=16, out=32, kernel=(2, 1), groups=16) BatchNorm2d(32), Activation (ELU/ReLU) AvgPool2d(kernel=(1, 4)), Dropout(p=0.25)
3. SeparableConv	Mixes temporal and spatial features across all feature maps efficiently.	Conv2d(in=32, out=32, kernel=(1, 15), padding=(0, 7)) BatchNorm2d(32), Activation (ELU/ReLU) AvgPool2d(kernel=(1, 8)), Dropout(p=0.25)

Block / Layer	Purpose	PyTorch Implementation & Filter Details
4. Classify	Flattens the feature maps and outputs class probabilities.	Flatten(), yields 736 features (32 * 1 * 23) Linear(in=736, out=2)

The kernel dimensions are shaped as (Channels, Timepoints) because EEG data is a 2D signal (Spatial/Channels ×Temporal Time).

- **EEGNet Temporal Kernel `(1, 51)` or `(1, 64)`:** The length of 51 or 64 timepoints acts as a bandpass frequency filter (Alpha or Beta rhythms). By looking at approximately 0.2 to 0.5 seconds of data at a time (depending on the sampling rate), the filter is long enough to capture the wave cycles of motor imagery without being distracted by localized noise. Again its design is to focus on observing the frequencies and that's why it is longer horizontally.
- **Spatial Kernel `(2, 1)`:** The dataset specifically has exactly 2 EEG channels. By setting the spatial kernel to height `2` and width `1`, the convolution spans across all available electrodes at a single point in time to learn the spatial relationship between the two sides of the brain.
- **Separable Kernel (1, 15):** After the initial aggressive pooling in EEGNet, the temporal resolution is heavily compressed. A smaller 15-point temporal kernel is sufficient to summarize the remaining time-series features before classification.
- **EEGNet F1=8 or 16 or 25 (Temporal Filters):** This dictates how many different frequency bands the network learns (e.g., one filter for 10Hz, another for 20Hz). The original paper recommends F1=8, while the TA spec uses F1=25. I keep this number under 20, because EEG signals might contain a few relevant frequency rhythms. Too many filters will just memorize noise and cause overfitting.
- **EEGNet D=2 (Depth Multiplier):** For every frequency filter, the network learns D=2 spatial filters. This maps to the physical brain: for a specific frequency, it can learn one spatial pattern for the "left hand" imagery and one for the "right hand" imagery.
- **EEGNet F2=16 or 32 or 50 (Pointwise Filters):** This is usually calculated as F1×D (e.g., 16×2=32). It mixes the combined temporal and spatial features without causing an information bottleneck before the final classification.
- **EEGNet Dropout (p=0.25):** The TA specification uses a light 25% dropout rate for EEGNet. Because EEGNet is designed to be compact, it helps resist overfitting. A smaller dropout like 0.25 provides just enough regularization to keep the network generalizing well without starving it of the important, limited information coming from the 1,080 training trials. Later I applied a lighter dropout (25%–30%) in the early spatial block, and a heavier dropout (50%) in the later separable block. The idea is to preserve as much raw, low level spatial correlation between the two EEG channels as possible early on, but penalize the high level features just before the classification layer to prevent them from memorizing the training set. However, what works best for my experiment is 0.5.

ELU Function Explanation

- **What it is:** The Exponential Linear Unit (ELU) is an activation function that outputs the input *x* if it is positive, and scales negative inputs using the formula $\alpha(e^{x} - 1)$.
- **Handling Negative Values:** Unlike ReLU, which completely zeroes out negative values, ELU allows negative values to pass through smoothly without deactivate the neuron, controlled by the α parameter.
- **Why we compared it:** EEG signals are noisy. We compared ELU against ReLU and LeakyReLU to see whether allowing negative background noise to pass smoothly (ELU) or forcing it to zero (ReLU/LeakyReLU) would yield better classification accuracy for motor imagery.

Depthwise & Separable Convolutions

- **Depthwise Convolution:** This layer applies spatial filters independently to each temporal feature map using the `groups` parameter. It learns optimal channel weights across the 2 EEG electrodes to find which parts of the brain contribute most, without mixing different frequency bands. Because it is kept from being fully connected across all channels, it significantly reduces parameters and prevents overfitting.
- **Separable Convolution:** This layer factorizes a standard convolution into two smaller steps: a depthwise temporal convolution (e.g., `kernel=(1, 15)`) and a pointwise `(1x1)` convolution. It merges the extracted temporal and spatial information from previous layers into a unified representation for final classification while drastically reducing parameter counts.

Experiment Findings

- **Why ReLU outperformed ELU (Noise Sparsity):** ReLU models achieved the highest accuracies (e.g., `88.33%`). Turning negative activations into 0, which effectively filters out low-amplitude background EEG noise. ELU performed worse because it preserved this noise, carrying it into the final classification layer.
- **The Necessity of Early Stopping & Checkpointing:** During training, the training accuracy quickly climbed to near ~99% while testing accuracy fluctuated and often declined. This clear sign of overfitting meant that keeping the final epoch's weights would result in poor generalization; saving the checkpoint at peak test accuracy (Early Stopping) was required.
- **Dropout on a Small Dataset:** Because the dataset is small, heavier dropout (e.g., `p=0.5` instead of the standard `p=0.25`) improved EEGNet's test accuracy. It forced the network to learn robust, generalized features rather than memorizing the specific training trials.
- **Number of Epoch:** For all EEGNet epochs are 700 and DeepConvNet are 3000. This is to help each model learn the data better since the size of the data is small.
- **Overfitting in Deep Models (DeepConvNet vs. EEGNet):** DeepConvNet contains over ~150,000 parameters compared to EEGNet's ~2,600. Even when applying extreme dropout (`0.75`), DeepConvNet rapidly overfit the training data (reaching ~99% train accuracy but only ~81-84% test accuracy). This proves that highly parameterized models are not suited for small EEG datasets, whereas the EEGNet handles the limited data well.

4. Github Link (5%)

- **Repository URL:** https://github.com/shem004/EEG_Classification_Lab2.git

5. Bonus (15%): DeepConvNet

- **Implementation Details:**

DeepConvNet Architecture Breakdown

Total Parameters: ~152,000

Block / Layer	Purpose	PyTorch Implementation & Filter Details
1. Block 1	Extracts initial temporal features, then immediately applies spatial filtering.	Conv2d(in=1, out=25, kernel=(1, 5)) Conv2d(in=25, out=25, kernel=(2, 1)) BatchNorm2d(25), Activation MaxPool2d(kernel=(1, 2)), Dropout(p=0.5)
2. Block 2	Extracts higher level temporal patterns from combined spatial features.	Conv2d(in=25, out=50, kernel=(1, 5)) BatchNorm2d(50), Activation MaxPool2d(kernel=(1, 2)), Dropout(p=0.5)
3. Block 3	Further downsamples and extracts deep temporal features.	Conv2d(in=50, out=100, kernel=(1, 5)) BatchNorm2d(100) -> Activation MaxPool2d(kernel=(1, 2)), Dropout(p=0.5)
4. Block 4	Final high-level feature extraction before classification.	Conv2d(in=100, out=200, kernel=(1, 5)) BatchNorm2d(200), Activation MaxPool2d(kernel=(1, 2)), Dropout(p=0.5)
5. Classify	Dense layer mapping deep features to 2 classes.	Flatten(), yields 8,600 features (200 * 1 * 43) Linear(in=8600, out=2)

- **DeepConvNet Temporal Kernels (1, 5), (2,1):** Instead of one massive time filter, DeepConvNet stacks multiple tiny (1, 5) temporal filters across 4 blocks. Stacking small kernels continuously downsamples the data to learn highly abstract, complex temporal

patterns at deeper layers. In further experiment I think I can enlarge the “Temporal Kernel” to (1, ~15-50) to catch longer frequency patterns but might overfit.

- **DeepConvNet (25 to 50 to 100 to 200 Filters):** DeepConvNet follows the traditional deep learning principle of doubling the number of feature maps as the spatial/temporal resolution is halved by Max Pooling. It starts at 25 filters to capture basic waves, and expands to 200 filters in the final block to capture complex combinations of those waves. Because 200 filters is massive for only 1,080 training trials, DeepConvNet requires large dropouts (p=0.5 to 0.75) to prevent overfitting.
- **Experiments & Accuracy:**

```
Early stopping triggered at Epoch 1676! Best Test Acc: 84.54% at Epoch 1256.
--> [DeepConvNet_LeakyReLU_0.10] Best Peak Test Acc: 84.54% (at Epoch 1256/1676)
```

- **Discussion:** DeepConvNet (~150k parameters) is prone to overfitting compared to EEGNet (~2.6k parameters) because the EEG data that we are working on is small(1080) and it requires higher dropout to regularize it, however results from DeepConvNet shouldn't beat EEGNet, as you can see in the experiment the training is close to ~100% meanwhile testing is dropping this sign that overfitting is happened.